

Deep Learning for NLP

Student name: Μπαρμπάρσος Θεοφάνης
sdi: sdi2200107

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Abstract	2
2	Data processing and analysis	2
2.1	Exploratory Data Analysis	2
2.2	Pre-processing	7
2.3	Data partitioning for train, test and validation	8
2.4	Vectorization	8
2.4.1	Understanding TF-IDF	8
2.4.2	TF-IDF Mechanism	8
2.4.3	Why Use log in IDF?	8
2.4.4	Limitations of TF-IDF	9
3	Algorithms and Experiments	9
3.1	Experiments	9
3.2	Hyper-parameter tuning	11
3.3	Optimization techniques	11
3.4	Evaluation	12
3.4.1	ROC Curve	12
3.4.2	Learning Curve	12
3.4.3	Confusion Matrix	12
4	Results and Overall Analysis	15
4.1	Results Analysis	15
4.1.1	Best trial	16
4.2	Comparison with the first project	17
4.3	Comparison with the second project	17
4.4	Comparison with the third project	17
5	Bibliography	17

1. Abstract

In this project, we developed a sentiment analysis classifier using Logistic Regression and TF-IDF feature extraction on an English-language Twitter dataset. The primary objective is to design an effective text classification model that can accurately predict the sentiment of a tweet as either positive or negative. This task involves several key steps, including exploratory data analysis, data preprocessing, feature extraction, model training, and evaluation using various performance metrics.

2. Data processing and analysis

2.1. Exploratory Data Analysis

In this first part of the project, I performed an Exploratory Data Analysis(EDA) of the given data, specifically, regarding both the training and the validation datasets. This process aims to provide a better view of the datasets as, through plots and various statistics we are able to better understand their composition.

1. Initially, the first 5 entries of each dataset are displayed as a brief sample of the dataset, along with some basic information describing the whole dataset such as, total number of entries, the different data types that make up each record etc.(Figure 1)

Figure 1:

---- Analysis for Training Dataset ----

	ID	Text	Label
0	189385	@whoisralphie dude I'm so bummed ur leaving!	0
1	58036	oh my god, a severed foot was foun in a wheely...	0
2	190139	I end up "dog dialing" sumtimes. Wha...	1
3	99313	@_rachelx meeeee toooooo!	0
4	157825	I was hoping I could stay home and work today,...	0

Basic info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148388 entries, 0 to 148387
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0    ID      148388 non-null    int64
1    Text     148388 non-null    object
2    Label    148388 non-null    int64
dtypes: int64(2), object(1)
memory usage: 3.4+ MB
```

2. Plots are printed to showcase the length distribution of tweets both in characters count and in words count. (Figures 2 & 3)

Figure 2:

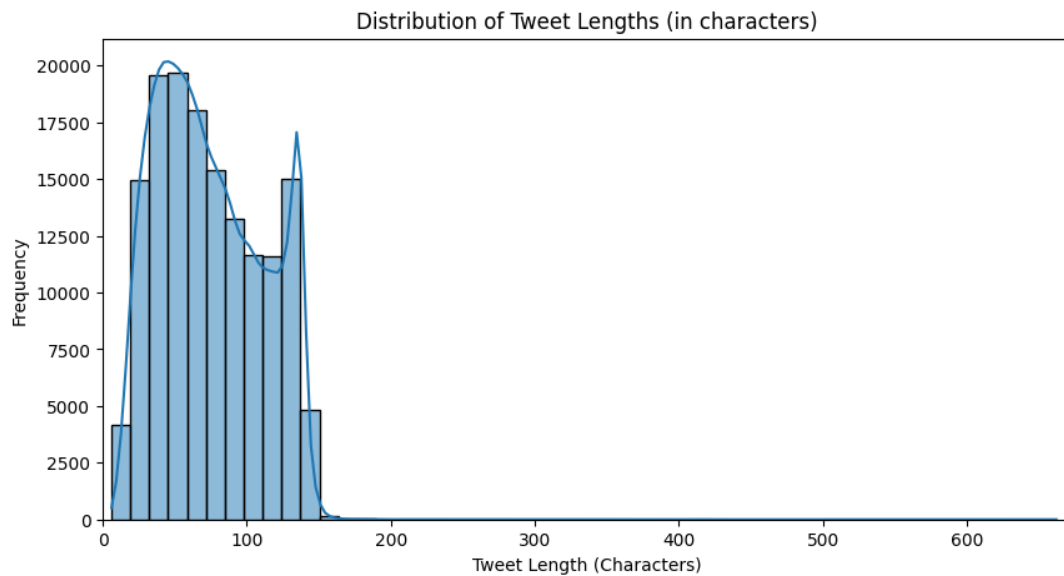
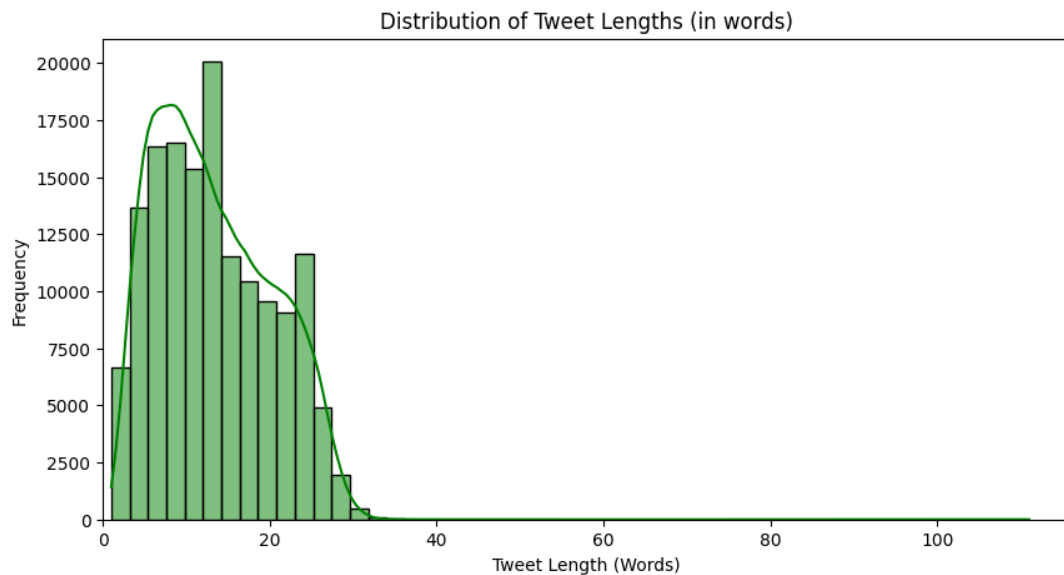
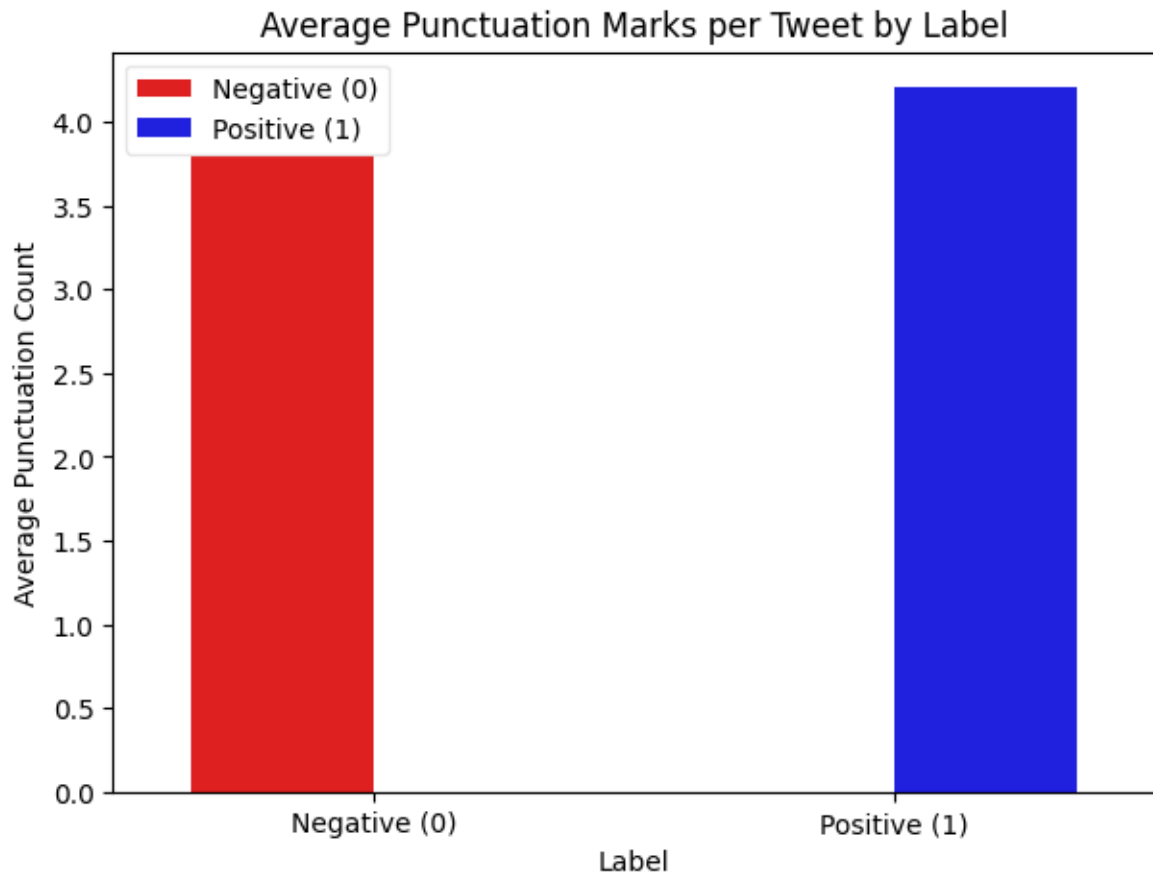


Figure 3:



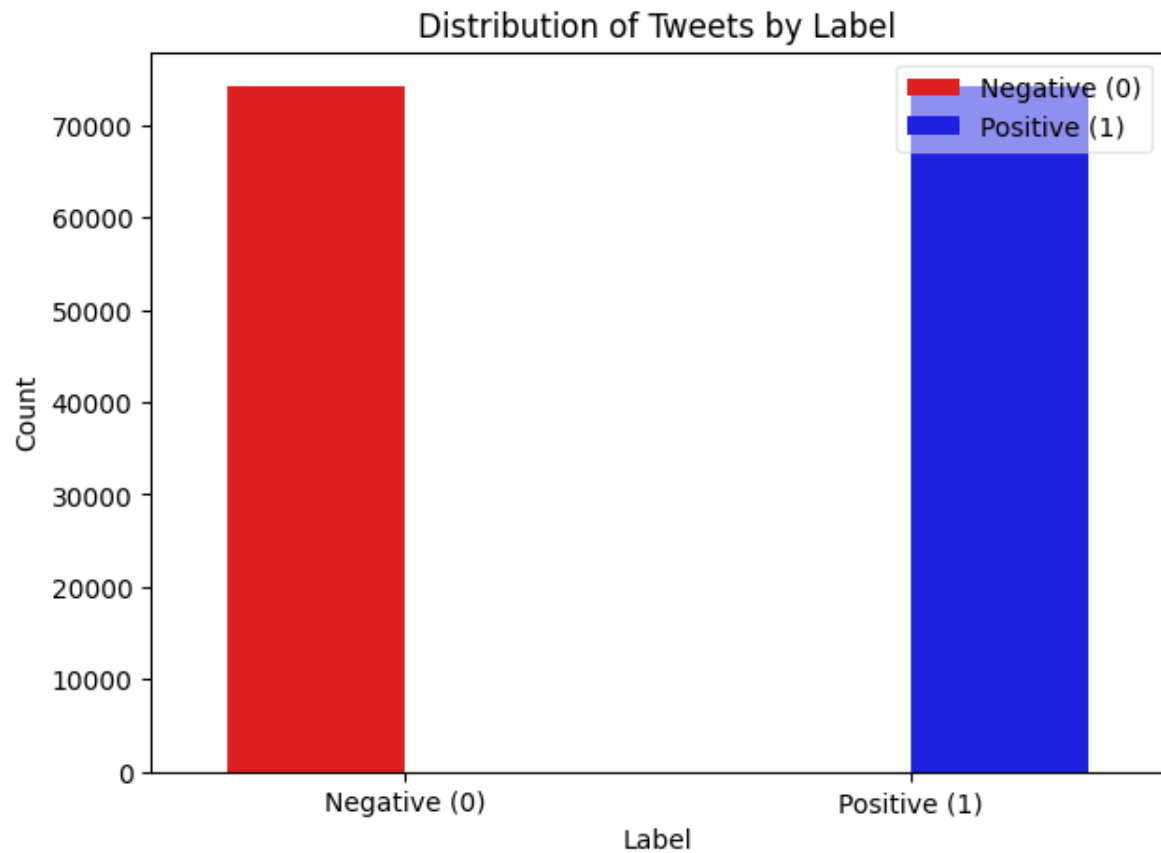
3. A plot is printed to showcase the number of punctuation marks in tweets per label. (Figure 4)

Figure 4:



4. A plot (alongside exact numbers) is printed to showcase the distribution of tweets per label. (Figure 5)

Figure 5:



Positive tweets in dataset: 74196

Negative tweets in dataset: 74192

5. Lastly, two plots are printed to showcase the number of correctly spelled words (Figure 6) and the number of misspelled words (Figure 7) in tweets per label.

Figure 6:

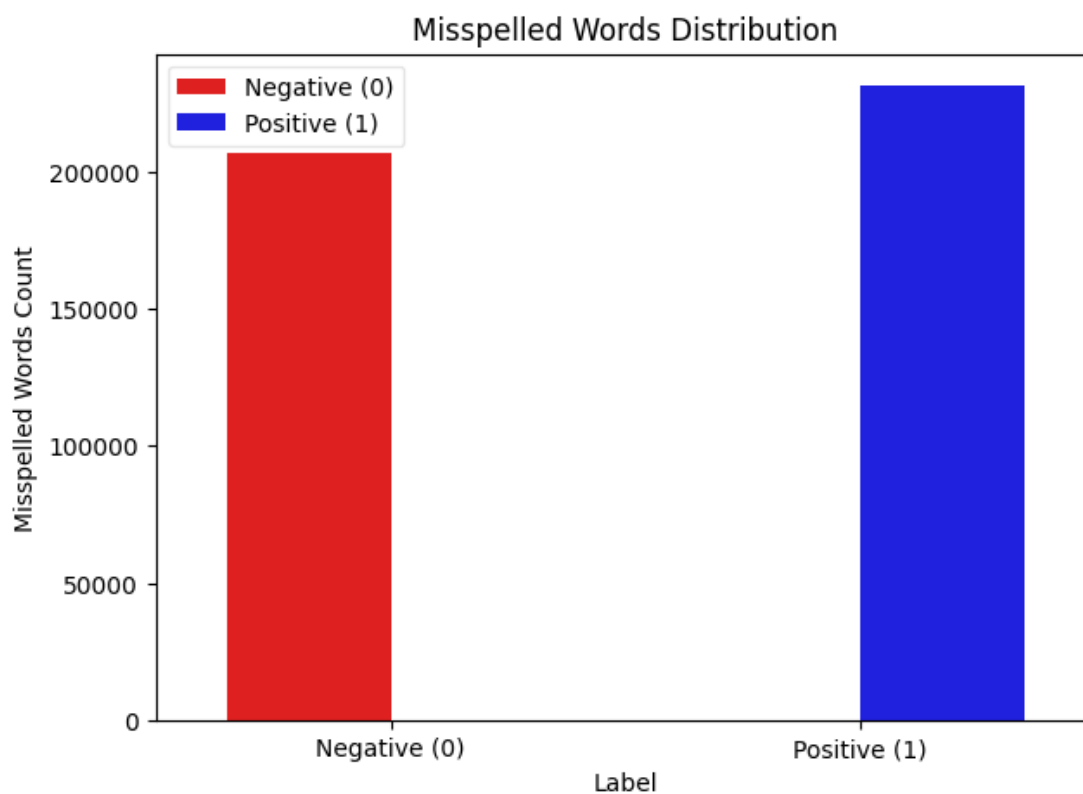
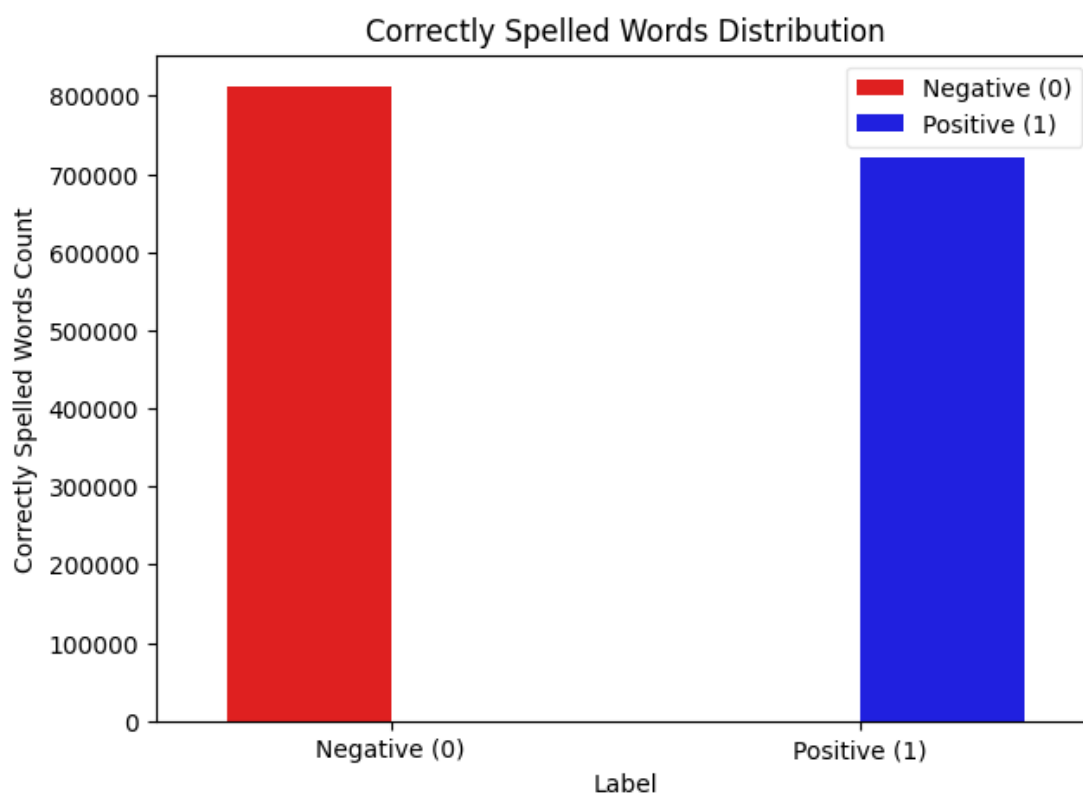


Figure 7:



As mentioned before, the EDA described above, is performed on both the training and the validation datasets. (specific plots and outputs refer to the training dataset as an example)

2.2. Pre-processing

In this step, I implemented multiple preprocessing techniques to clean and normalize the Twitter dataset, ensuring it was suitable for feature extraction and classification. The following methods were applied:

1. **Lowercasing:** All text was converted to lowercase to ensure consistency and eliminate redundancy in token representation.
2. **Replacing Emails, URLs, and Mentions:** Using regular expressions, email addresses were replaced with the token "email", URLs with "url", and Twitter mentions (e.g., @username) were removed, as they do not contribute to sentiment analysis.
3. **Handling Hashtags:** Hashtags were processed by removing the "#" symbol while preserving the associated word (e.g., "#happy" becomes "happy").
4. **Removing Ellipses:** Consecutive dots (e.g., "...", "...", ".....") were replaced with a single space to normalize punctuation usage.
5. **HTML Entity Decoding:** HTML-encoded characters were converted to their corresponding textual representation (e.g., "&" becomes "&").
6. **Handling Repeated Characters:** Words with unnecessarily long character repetitions were normalized by limiting repetitions to at most two occurrences (e.g., "soooo" becomes "soo").
7. **Expanding Contractions:** Common English contractions were expanded to their full forms (e.g., "haven't" becomes "have not") to enhance textual clarity and standardization.
8. **Removing Non-Alphanumeric Characters:** Any character that is not an English letter, number, or whitespace was removed to eliminate noise and improve text quality.
9. **Spell Checking:** A spell checker (e.g., PySpellChecker) was applied to correct misspelled words, ensuring more accurate text representation.

After applying these preprocessing techniques, the cleaned text was ready for feature extraction using TF-IDF. The cleaned dataset significantly improved classification performance by reducing irrelevant features and enhancing meaningful word representations.

The set of techniques mentioned above, in this specific order, is a choice conducted upon extensive testing and experimenting with various preprocessing methods.

2.3. Data partitioning for train, test and validation

The data for the project as a whole is partitioned in three datasets with the same structure: a training dataset, a validation dataset and a test dataset. Exploratory Data Analysis is performed on the training and validation datasets only as the test dataset is theoretically not given and it is used without ever knowing specifically what it contains. All three datasets undergo the same preprocessing as discussed previously. The model is then trained on the training dataset and is tested and evaluated on the validation dataset. Finally, a prediction result is produced for the test dataset which is used primarily for the Kaggle submission.

2.4. Vectorization

In this task, the **TF-IDF (Term Frequency-Inverse Document Frequency)** method was applied to convert textual data into numerical representations that are suitable for machine learning models.

2.4.1. Understanding TF-IDF. TF-IDF is a statistical approach used to determine the significance of a word in a document relative to an entire collection of documents. It highlights terms that appear frequently in a specific document but are uncommon across the dataset, allowing the model to focus on meaningful and distinctive words.

2.4.2. TF-IDF Mechanism. TF-IDF is composed of two key elements:

- **Term Frequency (TF):** Quantifies how often a word occurs in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in } d}$$

- **Inverse Document Frequency (IDF):** Assesses the significance of a word across the entire document set.

$$IDF(t) = \log \left(\frac{N}{DF(t)} \right)$$

where N represents the total number of documents, and $DF(t)$ denotes the number of documents containing the term t . The logarithm helps prevent highly frequent words from dominating the score.

The TF-IDF score is then computed as:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

2.4.3. Why Use log in IDF? The logarithmic function is used to ensure that extremely common words (such as "the" or "is") do not disproportionately affect the weighting. Without logarithmic scaling, the variance in IDF values would be significantly larger, leading to an unbalanced representation of term importance.

2.4.4. Limitations of TF-IDF. While TF-IDF is a powerful technique, it has certain drawbacks:

- It does not capture semantic relationships—synonyms like "happy" and "joy" are treated as unrelated words
- It disregards word order and phrases—terms such as "machine learning" are processed as independent words.
- It may assign excessive importance to rare words, even if they are irrelevant or misspelled.
- It does not differentiate between distinct meanings of the same word (e.g., "Apple" as a fruit versus "Apple" as a company).

3. Algorithms and Experiments

3.1. Experiments

To tackle the sentiment analysis problem, I started by experimenting with various preprocessing techniques to clean up the text data before training the model. Initially, I applied basic text-cleaning methods such as lowercasing and removing non-alphanumeric characters. However, I progressively refined my approach by testing more advanced techniques and analyzing their impact on model accuracy.

Preprocessing Trials

- **Stopwords Removal:** Contrary to common practice, removing stopwords decreased accuracy. Keeping stopwords led to better performance, likely because they contribute contextual information in sentiment analysis.
- **Spell Checking:** I tested multiple spell-checking methods:
 - Using an extensive dictionary with a wide range of possible misspellings did not significantly improve accuracy. Additionally, this approach is inefficient and will likely lead to overfitting.
 - TextBlob and SymSpell were both tested but did not result in better accuracy and ultimately opted for pyspellchecker to perform spell checking on the data.
 - Spell checking with distance=2 in the SpellChecker library was too slow (>30 minutes).
 - A hybrid spell-checking approach was also tested: first, all words were corrected using distance=1, and for words without corrections, distance=2 was applied. However, this was still too slow and did not improve accuracy.
- **Lemmatization & Stemming:** Both techniques unexpectedly reduced accuracy, likely due to the loss of important word forms that carry sentiment nuances.

- **Slang Normalization:** I attempted replacing common slang terms with their full equivalents. However, this approach reduced accuracy. The following function and dictionary was used:

```
def remove_slang(text):
    slang = {
        "gr8": "great", "gud": "good", "bbq": "barbeque",
        "btw": "by the way", "4all": "for all", "yeah": "yes",
        "cuz": "because", "coz": "because", "dis": "this",
        "dat": "that", "y'all": "you all", "nah": "no",
        "nope": "no", "omg": "oh my god", "thankz": "thanks",
        "btwn": "between", "cpl": "couple", "ty": "thank you",
        "imo": "in my opinion", "brb": "be right back",
        "ttyl": "talk to you later",
        "imho": "in my honest opinion",
        "fyi": "for your information", "tbh": "to be honest",
        "nvm": "nevermind", "w/o": "without",
        "wyd": "what are you doing", "ikr": "I know, right?",
        "ez": "easy", "prolly": "probably",
        "gimme": "give me", "lemme": "let me", "thanx":
        "thanks", "dunno": "do not know",
        "lmk": "let me know", "2night": "tonight",
        "2morrow": "tomorrow", "b2b": "back to back",
        "cya": "see you", "ya": "you", "lol": "laugh out loud",
        "lmao": "laughing my ass off",
        "lmfao": "laughing my fucking ass off",
        "rofl": "rolling on the floor",
        "roflmao": "rolling on the floor laughing my ass off",
        "streetz": "streets", "telly": "television",
        "tv": "television"
    }
    for wrong, correct in slang.items():
        text = re.sub(rf"\b{re.escape(wrong)}\b", correct, text)
    return text
```

When increasing the `n_gram_range` to (1,3) or (1,4), the negative impact of slang normalization was slightly reduced, but overall, avoiding slang replacement produced better accuracy.

- **Handling Time Expressions:** I experimented with the following regex pattern to normalize time expressions like "1030pm" to "1030 pm":

```
text = re.sub(r'(\d+)([ap]m)', r'\1 \2', text)
```

However, this preprocessing step led to a decrease in accuracy.

These trials highlight how different preprocessing techniques can significantly impact model performance. Some intuitive choices, such as lemmatization and slang normalization, unexpectedly reduced accuracy. The best results were achieved by retaining stopwords, avoiding excessive spell-checking, and carefully tuning text normalization techniques.

3.2. Hyper-parameter tuning

In order to optimize the performance of the Logistic Regression model, I manually experimented with various hyperparameters rather than utilizing tools like Grid Search, as it was computationally expensive and time-consuming.

The key hyperparameters I tuned were:

- **ngram_range:** I tested different values, including (1,2), (1,3), (2,3), (1,4), (1,5) and (1,6). The best results were obtained with (1,4), as it captured both individual words and meaningful phrases.
- **max_features:** After experimenting with values ranging from 1,000 to 1,000,000, I selected 100,000 as a balanced choice between performance and computational efficiency.
- **min_df and max_df:** Setting min_df=3 and max_df=0.9 helped filter out overly rare and overly common words, improving generalization.
- **Random State:** A fixed random_state=161 was used to ensure reproducibility.

The model was monitored for overfitting and underfitting:

- A very high training accuracy but poor validation accuracy indicated overfitting, which was mitigated by adjusting max_df and restricting the feature set.
- A low training accuracy indicated underfitting, which was improved by increasing max_features and tuning the ngram_range.

3.3. Optimization techniques

To further enhance model performance and training efficiency, the following optimization techniques were applied:

- **TF-IDF Weighting:** This technique ensured that words significant for classification were appropriately weighted, reducing the influence of frequently occurring but uninformative terms.
- **Feature Selection:** By limiting max_features and adjusting min_df, I removed noise from the dataset, improving generalization.
- **Regularization:** Logistic Regression inherently applies L2 regularization (Ridge), which helps in preventing overfitting by penalizing large coefficients.
- **Efficient Data Handling:** Sparse matrices were used to store TF-IDF vectors, reducing memory consumption and computational load.

These optimizations helped stabilize training and improved overall model robustness.

3.4. Evaluation

The model's performance was assessed using standard evaluation metrics:

- **Accuracy:** Measures overall correctness of predictions.
- **Precision, Recall, and F1-score:** Provides insights into false positives and false negatives.
- **Confusion Matrix:** Visualizes model misclassifications.
- **ROC Curve:** Evaluates the trade-off between true positive and false positive rates.

3.4.1. ROC Curve. The ROC (Receiver Operating Characteristic) curve illustrates the true positive rate against the false positive rate at various threshold levels. A higher AUC (Area Under the Curve) score indicates a better-performing model. The ROC curve is shown in Figure 8.

3.4.2. Learning Curve. The learning curve showcases training vs. validation accuracy over iterations. If there is a large gap between the two, it indicates overfitting. A properly tuned model should show convergence between these curves. (Figure 9)

3.4.3. Confusion Matrix. The confusion matrix (Figure 10) provides a breakdown of correct and incorrect predictions. It helps analyze where the model struggles (e.g., misclassifying positive sentiment as negative).

The matrix is as follows:

$$\begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix} \quad (1)$$

Where:

- **TP (True Positives):** Correctly predicted positive sentiments.
- **FN (False Negatives):** Positive sentiments misclassified as negative.
- **FP (False Positives):** Negative sentiments misclassified as positive.
- **TN (True Negatives):** Correctly predicted negative sentiments.

Analysis of these evaluation metrics, ensured model reliability and identified areas for further improvement.

Figure 8:

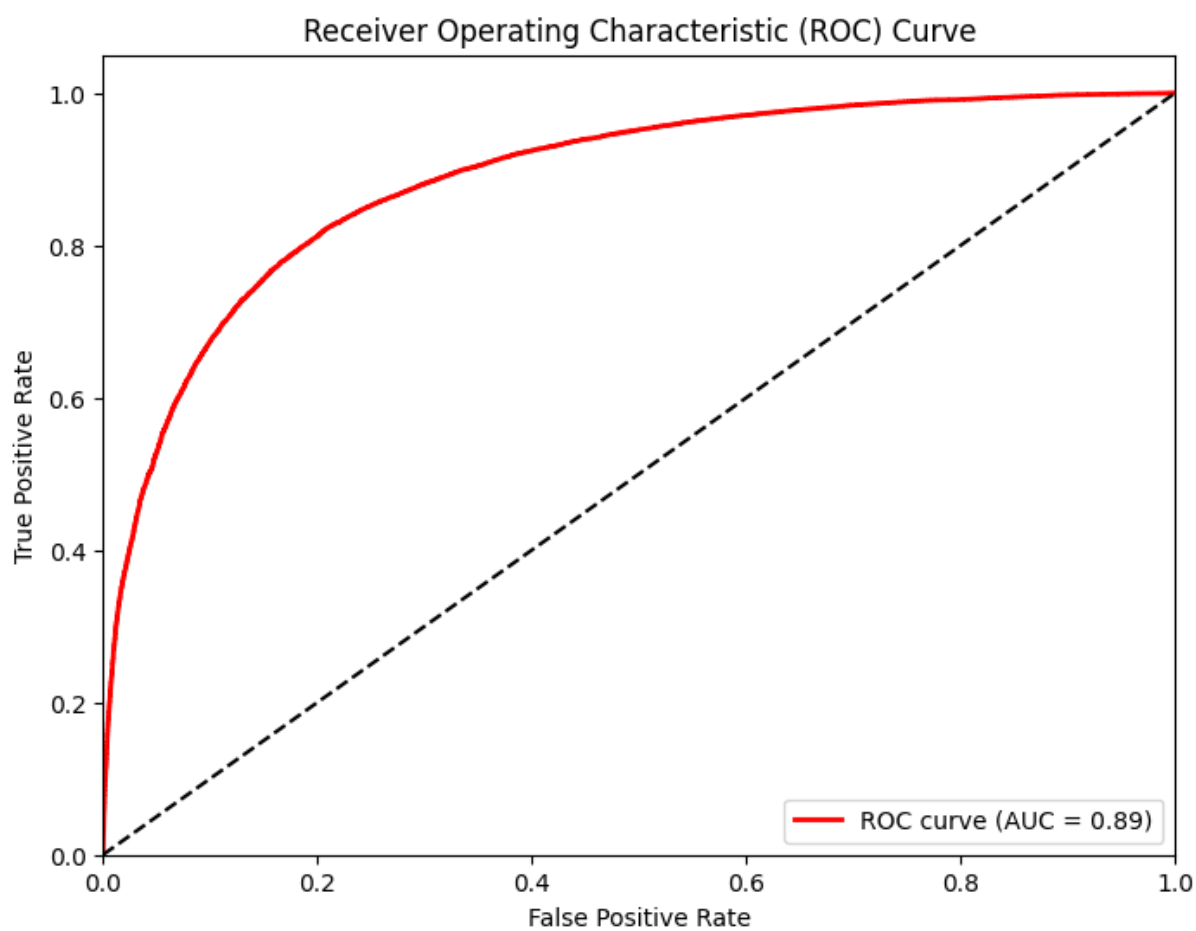


Figure 9:

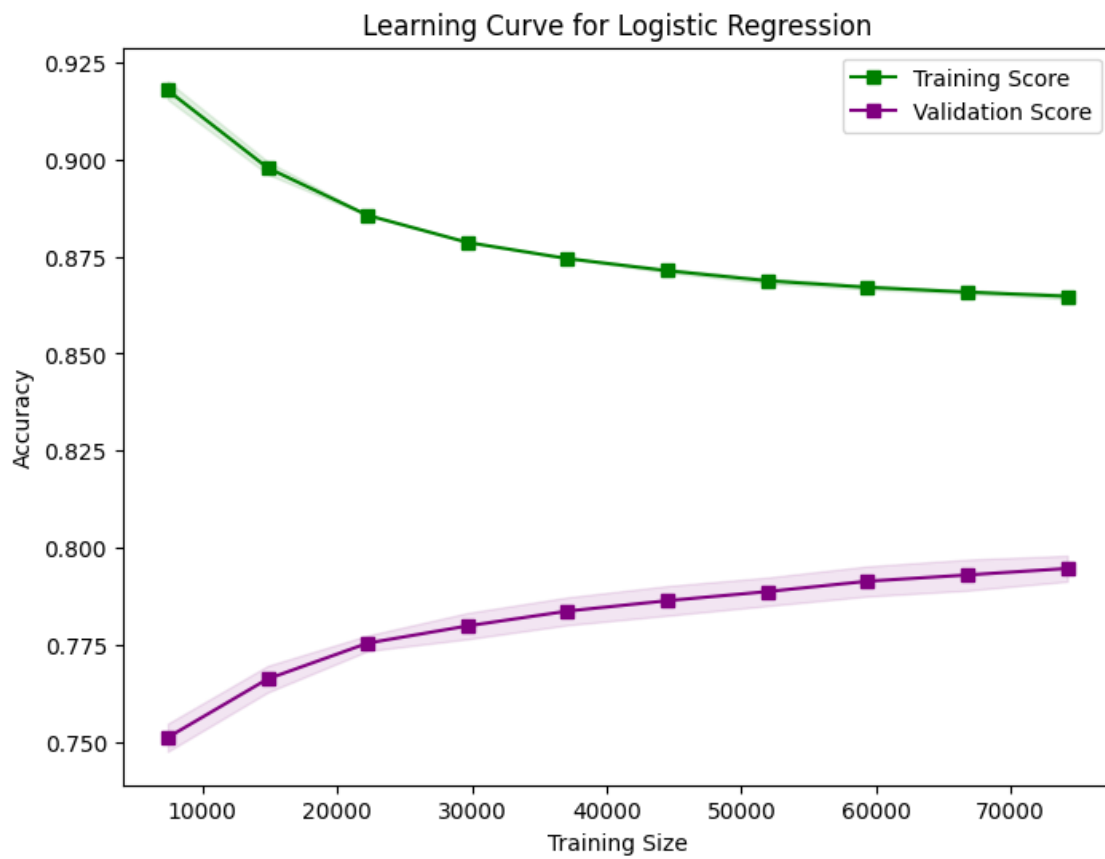
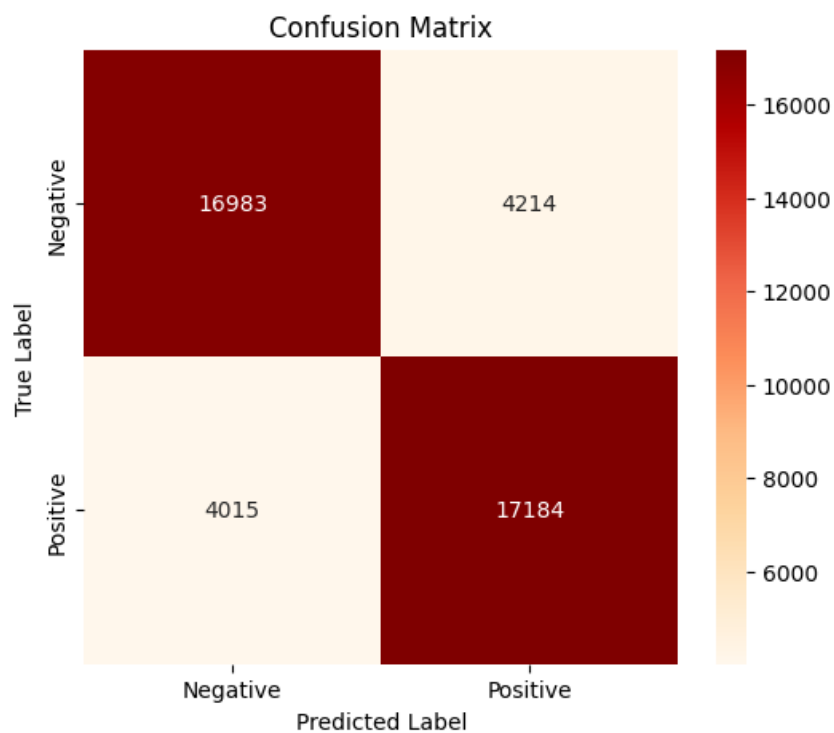


Figure 10:



4. Results and Overall Analysis

4.1. Results Analysis

The results of the project were both expected and somewhat surprising. While some outcomes were anticipated, some aspects did not align with our expectations. Specifically, some features I thought would improve the model's accuracy, actually had the opposite effect. As a result, various techniques were applied. Some of which improved performance as expected, while others did not, prompting further analysis and exploration of the model's decision-making process. This, in turn, led to a deeper understanding of what truly enhances the model and what does not.

Throughout this report, accuracy values are omitted because testing was performed locally. Therefore, specific numbers would not provide much useful information as the testing dataset and machine would ultimately differ from mine. Nonetheless, here are some results showcasing the process followed at various experimenting stages:

1. Only text lowercasing and the default arguments in the TfidfVectorizer() function call and only random_state=161 for the LogisticRegression(random_state=161): (Figure 11)
2. Full pre processing with all the pre processing techniques and all the hyperparameters discussed before: (Figure 12)

Figure 11:

Accuracy: 0.79113595622228516380					
Classification Report:					
	precision	recall	f1-score	support	
0	0.79	0.79	0.79	21197	
1	0.79	0.80	0.79	21199	
accuracy			0.79	42396	
macro avg	0.79	0.79	0.79	42396	
weighted avg	0.79	0.79	0.79	42396	

Figure 12:

Accuracy: 0.80653835267478068971					
Classification Report:					
	precision	recall	f1-score	support	
0	0.81	0.80	0.81	21197	
1	0.80	0.81	0.81	21199	
accuracy			0.81	42396	
macro avg	0.81	0.81	0.81	42396	
weighted avg	0.81	0.81	0.81	42396	

4.1.1. Best trial. My final, best trial is the one submitted to the competition. It incorporates all the preprocessing techniques specified in section 2.2 of this report, as well as all the hyperparameters for the `TfidfVectorizer()` and `LogisticRegression()` outlined in section 3.3. This combination of methods represents the optimal approach, ensuring the best possible text preparation and the most fine-tuned model training. Results from this trial (tested on the validation dataset and on my machine) are shown in Figure 12. However, as mentioned earlier, this is not entirely indicative of the actual performance on the competition platform. The final results I obtained on Kaggle are shown in Figure 13, with the best accuracy on the test dataset reaching 0.80484.

Figure 13:

Accuracy: 0.80590150014152273084

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.80	0.80	21197
1	0.80	0.81	0.81	21199
accuracy			0.81	42396
macro avg	0.81	0.81	0.81	42396
weighted avg	0.81	0.81	0.81	42396

4.2. Comparison with the first project

<Use only for projects 2,3,4>

<Comment the results. Why the results are better/worse/the same?>

4.3. Comparison with the second project

<Use only for projects 3,4>

<Comment the results. Why the results are better/worse/the same?>

4.4. Comparison with the third project

<Use only for project 4>

<Comment the results. Why the results are better/worse/the same?>

5. Bibliography

References

[1] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

<You should link and cite whatever you used or "got inspired" from the web. Use the / cite command and add the paper/website/tutorial in refs.bib>

<Example of citing a source is like this:> [1] <More about bibtex>